

Menu-Driven C Program for Digit-Based Operations

Using switch-case and separate void user-defined functions

1. Aim

To write, compile, execute, and test a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Each menu option is implemented using a separate user-defined function with void return type and a simple function call.

2. Problem Requirements

- Check whether a number is a Strong number.
- Check whether a number is an Armstrong number.
- Check whether a number is a Harshad number.
- Check whether a number is a Duck number.

Each function reads its required input, processes it, and displays the result. The main() function only displays the menu, reads the choice, and calls the selected function through switch-case.

3. Variables and Data Types

Variable	Data Type	Purpose
choice	int	Stores the selected menu option.
num, temp	int	Store the input number and a working copy.
digit	int	Stores the current digit.
sum	int	Stores digit/factorial/power sum.
fact	int	Stores factorial of a digit.
digits	int	Stores number of digits for Armstrong check.
i	int	Loop counter.

4. Algorithm

Main Program

1. Start.
2. Display the four menu options.
3. Read the user's choice.
4. Use switch-case to call strong(), armstrong(), harshad(), or duck().
5. For an invalid choice, display an error message.
6. Stop.

Strong Number - strong()

7. Read a positive integer num.
8. Copy num into temp and set sum = 0.
9. Extract each digit using digit = temp % 10.
10. Find factorial of the digit and add it to sum.
11. Remove the last digit using temp = temp / 10.
12. Repeat until temp becomes 0.
13. If sum equals num, display that it is a Strong number; otherwise display that it is not.

Armstrong Number - armstrong()

14. Read a positive integer num.
15. Count the number of digits and store it in digits.
16. Set sum = 0 and again process every digit.
17. Add digit raised to the power digits to sum.
18. If sum equals num, display that it is an Armstrong number; otherwise display that it is not.

Harshad Number - harshad()

19. Read a positive integer num.
20. Find the sum of all digits.
21. If num is exactly divisible by the digit sum, display that it is a Harshad number; otherwise display that it is not.

Duck Number - duck()

22. Read a positive integer num.
23. Examine every digit of the number.
24. If any digit is 0, display that it is a Duck number; otherwise display that it is not.

5. C Program

```
#include <stdio.h>
#include <math.h>

void strong(void);
void armstrong(void);
void harshad(void);
void duck(void);

int main(void)
{
    int choice;

    printf("DIGIT-BASED OPERATIONS\n");
    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            strong();
```

```

        break;
    case 2:
        armstrong();
        break;
    case 3:
        harshad();
        break;
    case 4:
        duck();
        break;
    default:
        printf("Invalid choice.\n");
}

return 0;
}

void strong(void)
{
    int num, temp, digit, sum = 0, fact, i;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    temp = num;

    while (temp > 0)
    {
        digit = temp % 10;
        fact = 1;
        for (i = 1; i <= digit; i++)
            fact = fact * i;
        sum = sum + fact;
        temp = temp / 10;
    }

    if (sum == num)
        printf("%d is a Strong number.\n", num);
    else
        printf("%d is not a Strong number.\n", num);
}

void armstrong(void)
{
    int num, temp, digit, digits = 0, sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    temp = num;

    while (temp > 0)
    {
        digits++;
        temp = temp / 10;
    }

    temp = num;
    while (temp > 0)
    {
        digit = temp % 10;
        sum = sum + (int)pow(digit, digits);
        temp = temp / 10;
    }

```

```

    }

    if (sum == num)
        printf("%d is an Armstrong number.\n", num);
    else
        printf("%d is not an Armstrong number.\n", num);
}

void harshad(void)
{
    int num, temp, digit, sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    temp = num;

    while (temp > 0)
    {
        digit = temp % 10;
        sum = sum + digit;
        temp = temp / 10;
    }

    if (num % sum == 0)
        printf("%d is a Harshad number.\n", num);
    else
        printf("%d is not a Harshad number.\n", num);
}

void duck(void)
{
    int num, temp, digit, found = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    temp = num;

    while (temp > 0)
    {
        digit = temp % 10;
        if (digit == 0)
        {
            found = 1;
            break;
        }
        temp = temp / 10;
    }

    if (found == 1)
        printf("%d is a Duck number.\n", num);
    else
        printf("%d is not a Duck number.\n", num);
}

```

6. Compilation and Execution

Compile the program using:

```
gcc hdoc18.c -o hdoc18
```

Run using:

```
./hdoc18
```

7. Execution and Test Outputs

```
(kali㉿kali)-[~/Desktop/apple]  
$ gcc hdoc18.c -o hdoc18  
  
(kali㉿kali)-[~/Desktop/apple]  
$
```

Compilation: Program compiled successfully without errors.

```
(kali㉿kali)-[~/Desktop/apple]  
$ ./hdoc18  
  
— Digit Based Operations —  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 1  
Enter a positive integer: 145  
145 is a Strong number.
```

Test 1 - Strong Number: Input 145

```
(kali@kali)-[~/Desktop/apple]  
$ ./hdoc18
```

— Digit Based Operations —

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 2

Enter a positive integer: 153

153 is an Armstrong number.

```
(kali@kali)-[~/Desktop/apple]
```

Test 2 - Armstrong Number: Input 153

```
(kali@kali)-[~/Desktop/apple]  
$ ./hdoc18
```

— Digit Based Operations —

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 3

Enter a positive integer: 18

18 is a Harshad number.

```
(kali@kali)-[~/Desktop/apple]
```

Test 3 - Harshad Number: Input 18

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc18

— Digit Based Operations —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer:
1023
1023 is a Duck number.

(kali㉿kali)-[~/Desktop/apple]
```

Test 4 - Duck Number: Input 1023

8. Result

The menu-driven C program was compiled and executed successfully. All four menu options were tested: 145 was identified as a Strong number, 153 as an Armstrong number, 18 as a Harshad number, and 1023 as a Duck number.